Parallelization in `loo` (current state)

Stan GSOC 2026

Table of contents

Guiding questions

- Q1: What is the effect of using mirai for serial execution (Section)?
- Q2: Can we use `mori` for sharing model methods (Section)?
- Q3: Where is parallelization currently in use in loo (Section)?
- Q4: Where could we benefit from introducing parallelization in loo (Section)?
- Q5: Where do we currently find nested parallelization in loo (Section)?

Background

> Flo: Probably here just super short the content of the GSoC project and a short outline (can also be added later). No need to go too much into the background (e.g., what is parallelization).

Please refer to Section for a glossary of some important terms.

Currently `loo` relies on the `parallel` package from base R [1] to parallelize certain computations, such as moment matching, Pareto smoothed importance sampling [2], and more. This works well on Unix systems, where `parallel` can use forking, but Windows requires a separate, socket based path. This project will replace `loo`'s current parallelization structure, swapping to a platform-agnostic approach using `mirai` [3]. This will have the added benefit of allowing users to parallelize execution over heterogenous compute resources (e.g., over local cores, a distributed systems, GPUs, or any mixture). This freedom decouples execution profiles from `loo`'s software, allowing users with varying compute needs to exploit their available resources without any extra work. Further, a major part of the project here would be writing strong documentation which teaches users how to make use of some common resources such as SLURM or SSH access to users, building off of the documentation in packages like `mirai`, but with a focus on applications to `loo`.

Approaches to parallelization

Currently, parallelization in loo goes through paths 1-3 [4]:

1. Serial execution using `lapply()`

- run one task after the other
- no parallelization; synchronous

2. Forking through `parallel::mclapply()`

- create child processes as a copy of the parent's memory that run in parallel, but the main R session blocks and waits for all of them to finish before returning the result list
- parallel, synchronous
- not available in Windows: falls back to `lapply()`

3. Fresh processes through `parallel::parLapply()`

- start worker processes as completely fresh R sessions, communicating with the controller over sockets. No shared memory which is in contrast to forking.
- parallel, synchronous

4. Parallelization with `mirai_map()` (**new**)

- dispatch tasks to persistent background R processes (daemons) over NNG sockets; workers stay alive between calls and are reused across tasks
- allows for asynchronous parallelization with persistent workers

Comparison between approaches

	`lapply()`	`mclapply()`	`parLapply()`	`mirai_map()`
Parallel	no	yes	yes	yes
Blocking	yes	yes	yes	yes (with `[]`)
Mechanism	in-process	forking	cluster (often PSOCK sockets)	NNG daemons
Shared memory	N/A	yes (copy-on-write)	no	no
Windows	yes	no	yes	yes
Persistent workers	no	no	yes	yes
Remote Execution	no	no	yes	yes
Export data	no	no	yes	yes
Setup needed	none	none	`makeCluster`	`daemons()`

Thus, `mirai` works for all OS and allows for asynchronous parallelization. The only major downside of `mirai` is that it does not provide a possibility for workers to share memory (however, when using `mirai` with only local daemons, one can use `mori`(see Section) [5]).

Introducing `mirai` for parallelization in `loo`

The parallelization package we are using is `mirai` [3], self described as a "Minimalist Async Evaluation Framework for R". Before we briefly describe how `mirai` works at a high level, it is useful to mention some of its benefits which led us to select it as our parallel backend.

`mirai` very naturally allows for heterogenous compute through a clean API. What this means in practice is that, once we migrate to `mirai`, users will be able to run `loo` code in parallel on their laptop, on remote computers, on a HPC system, or any zany mixture ([3], specifically see sec. 6 in the Reference Manual). Decoupling the execution profile from `loo` code, especially using the API `mirai` exposes, allows for users to more easily exploit their available computational resources without much hassle.

`mirai` is also widely used across major R packages–for example, recently landing in `purrr` ([6], [7], see `in_parallel()`). `mirai` is also "the first official alternative communications backend for base R's `parallel` package." [3], and is used in other major packages like `shiny` ([3], see the vignette), and in `tune`, which is a vital part of the `tidymodels` workflow ([8], see the vignette).

How does `mirai` work?

More on mirai internals, NNG and all that

Reproducibility with `mirai`

One thing worth discussing for a moment is random number generation in `mirai`, which uses the L'Ecuyer PRNG. The details around RNG in `mirai` are pretty straightforward, as we can see through a few very simple examples.

Firstly, even if we set the same seed, we won't get the same results from `lapply()` and `mirai_map()`. Here, we can see that if even if we use the same seed (`set.seed(0)` before each call), randomly generating a few numbers from an exponential distribution changes depending on whether we call `lapply()` or `mirai_map()`. More specifically, both maps are essentially doing `rexp(1), rexp(2), rexp(3), rexp(4)`.[1] In this snippet we are using 4 daemons, so in

[1] Somewhat orthogonal to this section, but note that `with(set.seed(0), lapply(1:4, rexp) |> unlist())` is equivalent to `with(set.seed(0), rexp(10))`.

the `mirai_map()` call, we are starting 4 RNG streams which are being derived from the main stream (see in source).

```
set.seed(0)
str(lapply(1:4, rexp))
```

```
List of 4
 $ : num 0.184
 $ : num [1:2] 0.146 0.14
 $ : num [1:3] 0.436 2.895 1.23
 $ : num [1:4] 0.54 0.957 0.147 1.391
```

```
# using with() just for scoping
with(
  {
    set.seed(0) # set seed in main process
    mirai::daemons(4) # start 4 background processes
  },
  # mirai_map() is an asynch map
  # `[]` blocks to collect results--see Glossary
  str(mirai::mirai_map(1:4, rexp)[])
)
```

```
List of 4
 $ : num 0.809
 $ : num [1:2] 0.63 0.285
 $ : num [1:3] 0.851 0.252 0.779
 $ : num [1:4] 1.603 0.705 0.249 0.618
```

Note that by default if `daemons()` is called without setting a seed, the RNG streams aren't reproducible (just like results in serial execution without setting a seed)–ensuring "statistical validity but not numerical reproducibility between runs" [9]. If we rerun the exact same `mirai` code, we do get the same results:

```
with(
  {
    set.seed(0)
    mirai::daemons(4)
  },
  str(mirai::mirai_map(1:4, rexp)[])
)
```

```
List of 4
 $ : num 0.809
 $ : num [1:2] 0.63 0.285
 $ : num [1:3] 0.851 0.252 0.779
 $ : num [1:4] 1.603 0.705 0.249 0.618
```

With 2 daemons, `mirai_map(1:4, rexp)` still creates four tasks: `rexp(1)`, `rexp(2)`, `rexp(3)`, `rexp(4)`. The first two tasks run on the two available daemons (so they have the same results as before); later tasks are assigned as daemons become free. Since RNG streams are tied to daemons, changing the number of daemons changes which stream each task uses, so the output changes.

```r
with(
  {
    set.seed(0)
    mirai::daemons(2)
  },
  str(mirai::mirai_map(1:4, rexp)[])
)
```

```
List of 4
 $ : num 0.809
 $ : num [1:2] 0.63 0.285
 $ : num [1:3] 0.636 1.804 0.756
 $ : num [1:4] 3.7305 0.731 0.0452 0.1211
```

Setting a seed manually when launching daemons gets reproducibility (stability over runs), "regardless of the number of daemons used." [9]. In this case, we are essentially seeding the daemons' processes directly instead of seeding the host session and making each daemon derive its RNG state from host.

```r
with(
  {
    mirai::daemons(4, seed = 0)
  },
  str(mirai::mirai_map(1:4, rexp)[])
)
```

```
List of 4
 $ : num 1.02
 $ : num [1:2] 0.0931 0.5214
```

```
 $ : num [1:3] 2.314 0.373 1.618
 $ : num [1:4] 0.821 0.886 3.642 1.572
```

```
with(
  {
    mirai::daemons(3, seed = 0)
  },
  str(mirai::mirai_map(1:4, rexp)[])
)
```

```
List of 4
 $ : num 1.02
 $ : num [1:2] 0.0931 0.5214
 $ : num [1:3] 2.314 0.373 1.618
 $ : num [1:4] 0.821 0.886 3.642 1.572
```

In essence, reproducibility is the users' responsibility, akin to compute setups. This further stresses the fact that documentation is a major part of this project.

Q1: Can we use `mirai` for serial execution?

Since `mirai` [3] neatly subsumes paths 2 and 3, a question we had was what would the performance hit be if we also removed 1–that is, if a user requested serial execution, what do we lose by using `mirai` with a single core.

```
log_ratios <- -loo::example_loglik_matrix()
S <- nrow(log_ratios)
N <- ncol(log_ratios)
tail_len <- loo:::n_pareto(r_eff = rep(1, N), S = S)

idx <- seq_len(N)

work <- function(i) {
  loo:::do_psis_i(
    log_ratios_i = log_ratios[, i],
    tail_len_i = tail_len[i]
  )$pareto_k
}

mirai::daemons(1)
```

```
results <- bench::mark(
  vapply = vapply(idx, work, numeric(1)),
  lapply = lapply(idx, work),
  mirai = mirai::mirai_map(
    idx,
    work,
    log_ratios = log_ratios,
    tail_len = tail_len
  )[],
  vapply_to_list = vapply(idx, work, numeric(1)) |> as.list(),
  purrr_map = purrr::map_dbl(idx, work),
  check = same_numeric_values,
  iterations = 200
)
```

Note that `mirai_map()` and `lapply()` results are lists. `same_numeric_values()`'s definition is elided from output but simply checks that results are identical without counting unlisting as part of the benchmark (unlike `vapply_to_list`).

```
results |> summary()
```

```
# A tibble: 5 x 6
  expression          min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>     <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 vapply           9.47ms   12.5ms      77.2    3.76MB   12.6
2 lapply           9.46ms   10.6ms      92.8    3.44MB   15.1
3 mirai           22.54ms   26.1ms      37.6  586.89KB    0.573
4 vapply_to_list    9.9ms   12.6ms      78.4    3.44MB   12.8
5 purrr_map        9.58ms   12.3ms      83.2     4.2MB   14.1
```

```
results |> summary(relative = TRUE)
```

```
# A tibble: 5 x 6
  expression       min median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>     <dbl>  <dbl>     <dbl>     <dbl>    <dbl>
1 vapply          1.00   1.17      2.05      6.56     21.9
2 lapply          1      1         2.47      6.00     26.4
3 mirai           2.38   2.45      1         1         1
4 vapply_to_list  1.05   1.19      2.08      6        22.3
5 purrr_map       1.01   1.16      2.21      7.33     24.6
```

NB: the `mem_alloc` column should not be interpreted as total memory usage for `mirai`: `bench::mark()` records R heap allocations via `utils::Rprofmem()`, which is not able to track daemons' memory usage [1], [3], [10].

`mirai_map()` with a single daemon has a median runtime of ~26ms, roughly 2.5x slower than the fastest in-process approach. Of course, in absolute terms the difference is negligible, adding only about 15ms. The memory usage of all in-process approaches are relatively close to one another. `vapply()`'s small memory overhead may probably be attributed to the extra work it does validating types and lengths and such [1].

In short, it is probably a good idea to directly test the effects of serial `mirai` execution in `loo`, but it is likely that we will find that we should keep the unique single core execution path.

Memory

Memory usage and `loo` (Why does it matter?)

Parallelization may lead to running out of memory (OOM). Copying the construction from that note: if you have, say, 8GB memory but spin up 16 tasks which each use 1GB, you will run out of memory. The math here is quite clear, though it is worst case and is complicated by the profile of the program–if that 1GB is peak usage and it normally doesn't go so high, and the peaks don't overlap, then it is possible for you to squeeze past without 16 gigs free. Note, however, that this 1GB usage per parallel worker is assuming that there is no memory that can be shared.

Shared memory & the potential of `mori`

`mirai` by itself does not allow for shared memory. However, `mori` is a recently released R package that allows for sharing memory across workers by placing an R object once into OS-level shared memory and letting every process on the machine read the same physical pages directly, with no data copying between processes [11].

If objects are reused across workers, we could using something like `mori` [11] in R to reduce the physical memory usage by mapping some objects in workers to the same shared address spaces, thus eliding copies–unless, of course, a worker modifies the object [5]. This can be loosely seen as mildly clawing back some of the benefits of forking with respect to memory. Note however, that `mori` works on a limited set of R objects, specifically "atomic vector types, lists, and data frames" [11]. See Section for empirical proof that we cannot naively use `mori` to share compiled model methods across workers.

Since OOM is a slightly tricky thing to think about, one user friendly feature we would like to implement, somewhat orthogonal to swapping to `mirai`, is having parallel functions check how

much memory they need and warn users if a run might run out of memory. We could do this pretty simply by running our functions across varying input sizes and modelling peak memory usage (probably averaged over a few runs) as a function of the parameters. Of course, if we implement this we would need to test it to see how accurate and conservative our model is.

Q2: Can we use `mori` for sharing model methods?

No.

In this example, we build a trivial model, compile model methods, and attempt to use `mori` to share the fit. As we can see, this doesn't work–even though `theta`, a simple vector, is actually shared. This just shows that `mori` is working, but `fit` cannot be shared [11].

```
library(cmdstanr)

stan_file <- file.path(
  cmdstan_path(),
  "examples",
  "bernoulli",
  "bernoulli.stan"
)

model <- cmdstan_model(
  stan_file,
  compile_model_methods = TRUE,
  force_recompile = TRUE
)

fit <- model$sample(
  data = list(N = 1, y = 0),
  fixed_param = TRUE,
  chains = 1,
  iter_warmup = 0,
  iter_sampling = 1,
  refresh = 0
)
```

```
Running MCMC with 1 chain...

Chain 1 finished in 0.0 seconds.
```

```
fit$init_model_methods()

theta <- mori::share(seq(0.01, 0.99, length.out = 100))
fit <- mori::share(fit)

mori::is_shared(fit)
```

```
[1] FALSE
```

```
mori::is_shared(theta)
```

```
[1] TRUE
```

```
lp <- function(theta, fit) {
  fit$log_prob(
    fit$unconstrain_variables(list(theta = theta)),
    jacobian = FALSE
  )
}

with(
  mirai::daemons(4),
  mirai::mirai_map(
    theta,
    lp,
    .args = list(fit = fit)
  )[] |>
    unlist() |>
    table()
)
```

```
Error in .Call(<pointer: (nil)>, ext_model_ptr, variables): NULL value passed as symbol addr
```

```
#> Error in .Call(<pointer: (nil)>, ext_model_ptr, variables): NULL value passed as symbol ad
#>
```

As we can see from the output of `mori::is_shared(fit)`, `fit` was not converted to a shared ALTREP form. `mori` successfully shares the plain numeric vector `theta`, but it does not share `fit`. The later `NULL` pointer errors arise when daemons try to call model methods through that non-shared `fit` object.

Q3: Where is parallelization currently in use in `loo`?

- `function()`
  - short description
- `function()`
  - short description

Q4: Where could we benefit from introducing parallelization in `loo`?

To investigate this question we use two approaches:

1) Inspecting the code base and figuring out at which points parallelization might be helpful ("analytic search")
2) Running performance tests on the loo pipeline, investigating where the bottlenecks are, and finding out whether we can improve things with parallelization.

There are some functions in loo which could benefit from parallelization, potentially to great effect, as these functions appear to do "embarrassingly parallel" work–meaning that they do independent tasks which seem amenable to speedups by executing those tasks concurrently. These as prime targets for parallelization and somewhat easily identified, we do so by inspection, as covered in the next section.

Analytic search

For each function/place in code where you think parallelization might help, do: * Introduction * short description of function * why do you think that parallelization can improve things? * Experiment: * implement parallelization * record time (define which time you are using) for parallel and unparallel version * report results (how much speed up we can get?) * Short conclusion

We can see in `elpd_loo_approximation()` that we don't push `cores` through to `loo.function()` so right now, this function only parallelizes if users set the `mc.cores` options as `loo.function()` will pick this up when its called without specifying `cores`. This is a super simple fix and could speed up some code, depending on how users tend to set their `cores`.

Later on in the same function, we can see that parallelization had been thought of but not implemented yet for the delta method approximations in that function (`waic_grad`, `waic_grad_marginal`, `waic_hess`). Each of these loop row-wise over data and shouldn't be too difficult to parallelize. More careful thought must go into determining if parallelization

is even worth it, and more careful scrutiny of the call sites of `elpd_loo_approximation()` to ensure it doesn't accidentally induce nested parallelization (see Section)–though at first blush this seems unlikely.

In `waic.function()` `loo` simply doesn't parallelize the work, relying only on a base `lapply()`. This seems like a pretty easy fix, and there doesn't seem to be any reason to not use parallelization. Again, should be careful and benchmark to ensure we aren't accidentally eating performance or RAM, but this could be a large, free win.

Lastly, in `ap_psis.array()`, which mostly just converts array arguments to matrices before passing the buck to the matrix method, we can note that cores gets hardcoded to 1 when calling `ap_psis.matrix()`. There doesn't seem to be any real reason to do this, eating the input `cores` like in the first example; unlike there though, we pass `cores = 1`. This is not a huge issue because users probably don't use this function, since most times they would be working directly with matrixes. Further evidencing this hypothesis is this line which should error out since it references a nonexistent `r_eff`. That obviously needs to be fixed, and this function tested.

Performance testing

- short description of approach for running performance test
- run performance test
- report results
- identify bottlenecks
- provide first ideas whether any of the identified bottlenecks can benefit from parallelization

Vectorization

When browsing the codebase to find places where parallelization could be helpful (see Section), we found two potential speedups–in these places, we may be able to benefit from *vectorizing* functions instead of parallelizing a loop. Vectorization is advantageous in these spots as we would be rewriting R loops as optimized matrix operations [12, Secs. 24.5–24.7].

> Matrix algebra is a general example of vectorisation. There loops are executed by highly tuned external libraries like BLAS. If you can figure out a way to use matrix algebra to solve your problem, you'll often get a very fast solution.

Of course, for these potential improvements, we should first write tests for correctness, then write benchmarks in the PR.

The more important of the two is in `pseudobma_weights()`, when performing a Bayesian bootstrap. This function is the backend for `loo_model_weights()`, and is used to average models using psuedo-Bayesian model averaging, optionally using a Bayesian bootstrap (which

is referred to as psudo-BMA+). This pseudo-BMA+ path is the one we care about as it has the elidable loop. We currently loop over Bayesian bootstrap iterations where it seems quite plausible to swap to matrix operations. This could potentially be a large speedup as the loop is large, being of length `BB_n`, which defaults to 1,000 replications. There don't appear to be any reallocations in the loop, so the speedup will probably be dominated by the efficiency of matrix ops [12, Secs. 5.3.1, 24.6]. This is tracked in #XXX.

Similarly, though less interesting, is a gradient calculation in `stacking_weights()`, just above the previous function. `stacking_weights()` is similarly used for combining models using stacking, that is, fitting a linear model to weigh the contending models. The gradient function is used for solving for those weights. However, the speedup here is less interesting because we are looping over `K`, the number of models being compared, which is usually small. However, we may as well take the (potential) free performance. This is tracked in #XXX.

These two loops seem like low hanging fruit and well worth the time to implement.

Nested parallelization

When updating `loo` we wish to avoid "nested parallelization", that is, attempting to compute something in parallel, when already in a parallel process. Take this nonfunctioning `mirai` code as an example:

```
# parallel over xs
mirai_map(
  xs,
  # parallel over x
  function(x) mirai_map(x, mean)[]
)
```

We first are doing something parallely over values of `xs`, but then we try to work in parallel again for each value from `xs`! The main problem here is oversubscription: we spawn `n` outer workers, let say, then for each we spawn another `m` inner workers. In the package context though, this implementation would be far more complex, and would probably be hidden completely from users. This becomes a problem when a user sets `n`, the outer parallel workers, to # cores - 1[2], as the actual number of parallel tasks could be as large as $n * m$. Oversubscription is bad because, in this context, we are mostly running CPU heavy tasks where each process takes time and needs its own core. Oversubscription is promising more parallelization than what the hardware can provide–note that parallelization usually has some fixed startup costs as well (see

[2]This is a common heuristic for number of threads/daemons/workers/etc. to use because it leaves 1 core free to do normal computer things, lest the system become unresponsive. Note this doesn't account for memory usage at all–if you have 8GB memory but send out n tasks, each using 2GB, you may run out of memory (see Section).

Section), so we would probably lose performance. In `loo` specifically, we must be careful when parallelizing functions to ensure we do not accidentally have nested parallelization. Note that `mirai` exposes a function, `on_daemon()`, which may be helpful to avoid nested parallelization [3]. However, this only accounts for nested `mirai` runs, and not nested parallelization on the whole.

Q5: Where do we currently find nested parallelization in `loo`?

- short description of function
- do you see any challenges with the identified functions?
- should nested parallelization run parallel or sequential?
  - provide experiments to stress your statement

Potential problems with packages that make use of `loo` such that nested parallelization can occur?

However, we know that other packages also call `loo` functions, which is another potential vector for nested parallelization. To this end, it seems like `brms` is the only (Stan) package where this could even be vaguely possible, but doing so seems to necessitate wilful parallelization not provided natively by `brms`. This merits further investigation, but it seems like we can (softly) table concerns regarding external nested parallelization for now.

Make a list of packages of which you have thought of: * package name * Any issues that you see? * (make a short note even if you don't see issues)

References

[1] R Core Team, *R: A language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing, 2026. Available: https://www.R-project.org/

[2] A. Vehtari, D. Simpson, A. Gelman, Y. Yao, and J. Gabry, "Pareto smoothed importance sampling," *Journal of Machine Learning Research*, vol. 25, no. 72, pp. 1–58, 2024, Available: http://jmlr.org/papers/v25/19-556.html

[3] C. Gao, *Mirai: Minimalist async evaluation framework for r*. 2026. Available: https://mirai.r-lib.org

[4] A. Vehtari *et al.*, "Loo: Efficient leave-one-out cross-validation and WAIC for bayesian-models." 2025. Available: https://mc-stan.org/loo/

[5] C. Gao, *Mori: Shared memory for r objects*. 2026. Available: https://shikokuchuo.net/mori/

[6] D. V. Charlie Gao Hadley Wickham and L. Henry, "Parallel processing in purrr 1.1.0," July 10, 2025. Available: https://tidyverse.org/blog/2025/07/purrr-1-1-0-parallel/

[7] H. Wickham and L. Henry, *Purrr: Functional programming tools*. 2026. Available: https://purrr.tidyverse.org/
[8] M. Kuhn, *Tune: Tidy tuning tools*. 2026. Available: https://tune.tidymodels.org/
[9] C. Gao, "Mirai 2.5.0," Sept. 05, 2025. Available: https://tidyverse.org/blog/2025/09/mirai-2-5-0/
[10] J. Hester and D. Vaughan, *Bench: High precision timing of r expressions*. 2025. Available: https://bench.r-lib.org/
[11] C. Gao, "Mori: Shared memory for r objects," Apr. 23, 2026. Available: https://opensource.posit.co/blog/2026-04-23_mori-0-1-0/. [Accessed: May 26, 2026]
[12] H. Wickham, *Advanced r*, 2nd ed. Chapman; Hall/CRC, 2019. doi: 10.1201/9781351201315. Available: https://adv-r.hadley.nz/
[13] L. Walthert and J. Wujciak-Jens, *Touchstone: Continuous benchmarking with statistical confidence based on 'git' branches*. 2026. Available: https://github.com/lorenzwalthert/touchstone

Appendix {.unnumbered .unlisted}

Glossary

Concepts

Serial

Tasks run one at a time, strictly one after another with no overlap. Always implies synchronous execution.

Synchronous

The calling code blocks and waits for a result before continuing. Does not imply serial as tasks may run in parallel while the caller waits. Example: Parallel sampling across 4 chains but before we can continue, we have to wait until all 4 chains are finished.

Asynchronous

The calling code moves on immediately after dispatching a task and can check back later for results. Enables non-blocking workflows.

Parallel

Multiple tasks run at the same time across separate workers. Can be combined with either synchronous or asynchronous calling patterns. Examples:

- Synchronous parallel: running 4 chains in parallel but blocking until all are done.
- Asynchronous parallel: dispatching multiple model fits to background processes. The main session fires off each model fit without blocking, so you could immediately dispatch the next model. Each background process then internally runs its 4 chains in parallel (sync).

Forking

A child process is created by copying the parent R session at the OS level. The child inherits the full environment (packages, variables) via copy-on-write memory. Unix/macOS only.

PSOCK cluster

Worker processes started as completely fresh R sessions, communicating with the controller over sockets. No shared memory; packages and variables must be exported explicitly.

Daemon

A persistent background R process that stays alive between tasks, waiting to receive and execute work. Used by `mirai` as its worker model.

Persistent workers

A worker process that remains alive between tasks and across multiple calls, retaining its state and loaded packages. Avoids the startup cost of spawning a new process per job. Contrast with temporary workers (e.g. forked processes in `mclapply()`) that are created for a single call and exit when it finishes.

Functions

`lapply()`

Base R serial map. Applies a function to each element of a list sequentially in the host session. No workers, no overhead; considered as the baseline.

`mclapply()` (Unix only)

Parallel map via forking. Workers inherit the session environment automatically. Falls back to `lapply()` (i.e., serial execution) on Windows.

`parLapply()` (all OS)

Parallel map over a PSOCK cluster. Requires explicit setup (`makeCluster`), export of variables, and teardown (`stopCluster`). Workers persist until stopped.

`daemons()`

Launches n persistent background workers for the current `mirai` session. Workers stay alive and idle until reassigned or the session ends. Workers can run on the same machine, or can be remote resources, and roughly can be considered as background R processes. See the documentation for more details.

`mirai_map()`

Parallel async map that dispatches tasks to daemons. Returns a list of unresolved mirai objects immediately. Append `[]` to block and collect all results.

`[]`

The collect operator on a mirai result. Blocks the main session until all dispatched tasks are complete and returns their values. Makes an async call synchronous.

`b3`: Bayesian Branch Benchmarking

What follows is a proposal for `b3`, a benchmarking tool designed to replace `touchstone` in Stan R packages and is far more flexible and broader in scope. `b3` is included here in the appendix as it could potentially improve our PR-level performance benchmarking, both for time and memory usage.

`b3` aims to measure language-agnostic end-to-end branch-vs-baseline performance diffs. `b3`'s philosophy is akin to touchstone [13], but shucks the R focus–`b3` handles orchestration, measurement, and reporting, while users must provide their project runtime, dependency setup, and benchmark command. `b3` compares complete repository states: if the candidate branch changes the benchmark, data, config, or dependencies, that change is part of the measured result.

Core

`b3` is one prebuilt native binary.

Users only set up their project runtime and declare their benchmarks.

R users set up R. Node users set up Node. Rust users set up Rust. Python users set up Python.

Setup and benchmark commands are opaque subprocesses specified by the user which `b3` simply executes. On Unix they may be Bash; on Windows they may be PowerShell; for R projects they may be `Rscript` directly.

`b3` handles:

git worktrees runtime preset environment variables optional setup commands paired randomized benchmark runs timing process-tree peak memory polling Bayesian bootstrap posterior summaries

Evaluation Environment

Similar to `touchstone`, `b3` separates environment isolation from dependency caching. Each variant gets its own environment inside its worktree, so baseline and candidate can resolve different dependency graphs safely. Download caches are shared across variants and CI runs, so unchanged packages do not need to be downloaded or rebuilt repeatedly. Of course, `b3` does not resolve dependencies itself–it exposes runtime presets that configure standard env/cache locations, then runs the user's setup command. Users can always opt into specifying everything manually. Regardless, dependency resolution remains the responsibility of the project's normal tooling: `rv`, `renv`, `pak`, `uv`, `npm`, `cargo`, etc.

`b3` can optionally run a cleanup command before each measured run. By default, dependency environments are reused across runs, but benchmark-created state is the user's responsibility. Benchmarks should either avoid persistent side effects which impact performance, or configure cleanup so each measured run starts from a comparable state.

Measurement

`b3` can run one or more named benchmarks. Each benchmark gets its own paired randomized baseline/candidate runs and its own posterior summaries for time and memory usage. For each paired block, randomize order: baseline then candidate, or candidate then baseline. Each command runs from its own worktree.

`b3` records:

elapsed_sec average_memory_bytes peak_memory_bytes exit_status

Runs with non-zero exit status are reported separately and excluded from posterior summaries unless explicitly configured otherwise. Note that memory measurement is best-effort and potentially platform-dependent. Short-lived spikes, detached subprocesses, permission boundaries, and OS-specific memory accounting may affect the reported value. Limitations/scope will be made abundantly clear in the API.

Analysis

For each metric, B3 computes paired log ratios:

$$d_i = log(candidate_i / baseline_i)$$

B3 uses a Bayesian bootstrap over paired blocks: each posterior draw samples Dirichlet weights over blocks, computes the weighted mean of the paired log ratios. B3 also employs mild shrinkage toward 0 (which can be disabled).

B3 reports:

median candidate/baseline ratio median percent change credible intervals P(candidate worse) P(candidate >1%, >5%, >10%, >20% worse)

B3 also reports order-stratified posterior summaries for baseline-first and candidate-first paired blocks. If the order-stratified effects disagree materially, this *may* mean that the results are unstable and that the number of runs should be increased. The benchmarks/commands may be suspect as well. In the future, B3 might expand to a more complex model which can better capture order effects.

Misc.

`b3` will also help scaffold GHA workflows to quickstart using `b3` for PR-level performance reports. Extensions to the project include a more complex model, wrappers around the core CLI in various host languages to facilitate usage, first-class support for standing up non-GHA CI/CD workflows, and so on.

Session Info

```
sessioninfo::session_info(
  pkgs = c("loo", "mirai", "bench", "purrr", "sessioninfo"),
  info = c("platform", "packages"),
  dependencies = FALSE
)
```

```
- Session info -------------------------------------------------------------
 setting  value
 version  R version 4.6.0 (2026-04-24 ucrt)
 os       Windows 11 x64 (build 26200)
 system   x86_64, mingw32
 ui       RTerm
 language (EN)
 collate  English_United States.utf8
 ctype    English_United States.utf8
 tz       America/Los_Angeles
 date     2026-06-06
 pandoc   3.8.3 @ c:\\Program Files\\Positron\\resources\\app\\quarto\\bin\\tools/ (via rmarkd
 quarto   1.9.36 @ C:\\Users\\visru\\AppData\\Local\\Programs\\Quarto\\bin\\quarto.exe

- Packages -----------------------------------------------------------------
 package     * version    date (UTC) lib source
 bench         1.1.4      2025-01-16 [1] RSPM (R 4.6.0)
 loo           2.9.0      2025-12-23 [1] RSPM (R 4.6.0)
 mirai         2.7.0      2026-05-08 [1] RSPM (R 4.6.0)
 purrr         1.2.2      2026-04-10 [1] RSPM (R 4.6.0)
 sessioninfo   1.2.3.9000 2026-06-02 [1] Github (r-lib/sessioninfo@d4e98a4)

 [1] C:/Users/visru/AppData/Local/R/win-library/4.6
 [2] C:/Program Files/R/R-4.6.0/library

------------------------------------------------------------------------------
```